

Compressor d'Arxius basat en Huffman: Enfocament Àvid i Gestió Eficient de Memòria

Josep Oliver Vallespir, Hugo Valls Sabater

Resum—Aquest document presenta el disseny i la implementació d'un compressor d'arxius sense pèrdua utilitzant l'algorisme de codificació de Huffman de longitud variable. S'hi detalla l'aplicació d'un enfocament àvid (greedy) per minimitzar l'entropia de la informació, les estratègies arquitectòniques per a una gestió eficient de la memòria mitjançant fluxos de dades, i el disseny d'una interfície d'usuari interactiva basada en el patró Model-Vista-Controlador (MVC).

Index Terms—Compressió sense pèrdua, Codificació de longitud variable, Algorismes Àvids, Huffman, Entropia de Shannon, Codis lliures de prefix, Concurrencia, MVC, Fluxos de dades, Binary Heap, Fibonacci Heap, Complexitat algorítmica.



1 Introducció

L'emmagatzematge i la transmissió eficient d'informació són pilars fonamentals en el desenvolupament de programari modern. Amb l'augment exponencial del volum de dades, la compressió sense pèrdua (*lossless*) esdevé una eina indispensable. En aquesta pràctica, s'aborda el disseny i la implementació d'un compressor d'arxius basat en l'algorisme de codificació de Huffman de longitud variable, orientat a la mínima entropia.

El problema principal que resol aquest projecte és la reducció de l'espai que ocupa un fitxer aprofitant la redundància de les seves dades. Per aconseguir-ho, s'aplica l'algorisme de Huffman, un mètode clàssic que s'emmarca dins de la família dels **algorismes àvids** (o de "curta mira"). L'enfocament àvid es caracteritza per prendre decisions localment òptimes en cada iteració amb l'esperança d'assolir un òptim global. En el nostre context, això es tradueix en la construcció d'un arbre binari on, pas a pas, s'uneixen els dos nodes (símbols o subarbres) amb la freqüència d'aparició més baixa. Aquesta estratègia garanteix l'assignació de codis binaris més curts als caràcters més freqüents (reduint l'entropia del missatge) i codis més llargs als menys freqüents, obtenint així la compressió òptima per a una distribució de dades concreta.[1]

Més enllà de l'algorítmia pura, el projecte planteja un repte crític a nivell d'enginyeria de programari: la **gestió eficient de la memòria**. Per tal que l'aplicació sigui viable en entorns reals, on els fitxers poden pesar

desenes o centenars de megabytes, s'ha descartat la càrrega íntegra de l'arxiu a la memòria RAM. En el seu lloc, el sistema s'ha dissenyat per processar la informació mitjançant fluxos de dades (*streams*), llegint i escrivint blocs de bytes. Això manté la complexitat espacial constant respecte a la mida del fitxer, prevenint errors de desbordament de memòria (*OutOfMemoryError*).

Finalment, tot el sistema s'ha estructurat seguint el patró de disseny **Model-Vista-Controlador (MVC)**, garantint una separació nítida entre la lògica de compressió, la gestió d'esdeveniments i la interfície gràfica d'usuari.

2 Arquitectura del Programa i Patró MVC

2.1 Separació de Responsabilitats

El sistema s'ha estructurat seguint rigorosament el patró **Model-Vista-Controlador (MVC)**, que estableix una separació estricta entre tres capes. Aquesta arquitectura permet que la lògica de compressió sigui totalment independent de la representació gràfica.

- **Model (modelo):** Conté el nucli algorítmic del projecte. En aquesta versió final, el Model no només gestiona l'arbre de Huffman, sinó que encapsula la lògica de persistència de dades mitjançant les classes `LectorBits` i `EscritorBits`. Aquesta abstracció permet llegir i escriure fitxers `.huff` a nivell de bit sense que les altres capes hagin de gestionar la complexitat del buffer de memòria.
- **Vista (vista):** Construeix i gestiona tots els elements Swing. Proporciona una interfície interactiva per a l'usuari. Com a novetat en aquesta

• *Estudiants del Grau en Enginyeria Informàtica a la Universitat de les Illes Balears (UIB).*
Assignatura: Algorismes Avançats.

pràctica, la Vista inclou un component especialitzat, el `PanelArbreHuffman`, que permet la visualització gràfica de l'estructura de dades generada pel Model, facilitant la depuració i la comprensió de la codificació.

- **Controlador (controlador):** Actua com a intermediari, gestionant el flux d'esdeveniments. Un detall crític implementat en aquesta fase és la gestió del multi-threading: el Controlador orquestra l'execució de la compressió en fils de treball secundaris per evitar el bloqueig de la interfície d'usuari (EDT), utilitzant la interfície `IProgresListener` per actualitzar la barra de progrés de la Vista en temps real mentre el Model processa les dades.

A continuació es mostra l'estructura de carpetes del programa seguint aquest patró:

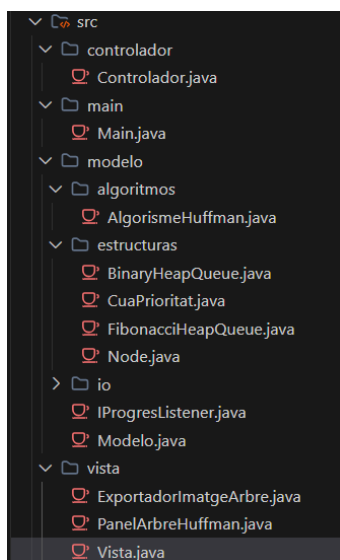


Figura 1. Esquema de classes del programa.

2.2 Comunicació entre Capes: el Patró Listener

Un dels reptes ha estat notificar el progrés des del Model cap a la Vista sense acoblar les capes. S'ha resolt mitjançant la interfície `IProgresListener`. El Controlador implementa la lògica d'actualització i la injecta al Model, permetent que aquest sigui totalment reutilitzable en altres entorns sense dependències visuals.

2.3 Concurrencia i Thread Safety

Atès que la compressió de fitxers de gran volum pot trigar diversos segons, s'ha posat especial èmfasi en la gestió de fils:

- **Separació de fils:** Les operacions pesades s'executen en un *Worker Thread*, deixant l'EDT lliure per mantenir la GUI reactiva.
- **Thread Safety:** Les actualitzacions visuals es redirigeixen a l'EDT mitjançant `SwingUtilities.invokeLater()`.
- **Mecanisme de cancel·lació:** S'utilitza una bandera `volatile` per permetre al fil de la GUI aturar el fil de compressió de forma segura i immediata.

3 Disseny de la Interfície d'Usuari (Vista)

La interfície gràfica (GUI) ha estat desenvolupada utilitzant la biblioteca *Swing* de Java[4], seguint els principis de disseny centrat en l'usuari. L'objectiu ha estat crear un entorn de treball net, professional i que permeti monitorar en tot moment l'estat del procés de compressió. La pantalla principal es divideix en tres grans àrees funcionals:

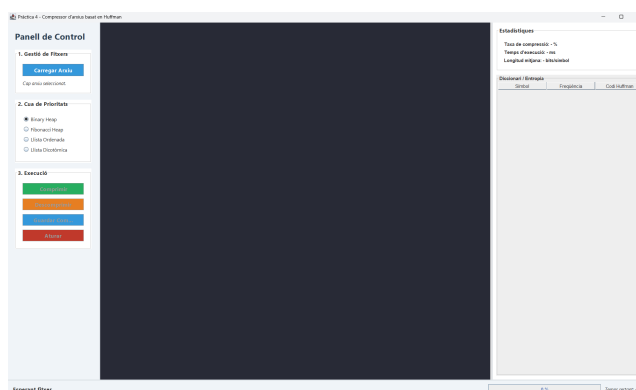


Figura 2. GUI Principal del Programa.

3.1 Panell Lateral (Control i Configuració)

Aquest panell, situat a l'esquerra de la finestra, actua com el centre de comandament de l'aplicació. S'ha dividit en tres targetes visuals per guiar l'usuari lògicament a través del procés:

- **Gestió de Fitxers:** Incorpora un botó per obrir l'explorador d'arxius (`JFileChooser`). Un cop seleccionat l'arxiu, es mostra el seu nom i, de manera dinàmica, el seu pes exacte (traduït a KB, MB o GB). Conèixer el pes inicial és vital per donar context a la futura taxa de compressió.
- **Cua de Prioritats:** Per donar compliment als requisits addicionals, s'ofereix un selector d'estructures de dades *Binary Heap* i *Fibonacci Heap* (possibilitat d'ampliar a més estructures de dades com *Llistes*, les quals no s'han implementat).

per aquesta entrega). A n'aquestes estructures s'emmagatzemaran els nodes abans de fusionar-los i també ens serviran per comparar rendiments de compressió.

- **Execució:** Conté els botons d'acció principals ("Comprimir", "Descomprimir", "Guardar Com..." i "Aturar"). Aquests botons romanen inactius fins que el sistema valida que s'ha carregat un fitxer correctament, evitant errors d'execució o accions que en moments determinats no es poden activar.

3.2 Panell Central (Visualització i Estadístiques)

L'àrea de major mida de la finestra està dedicada a la comprensió visual de l'algorisme. S'ha implementat un component divisor (JSplitPane) que permet a l'usuari redimensionar dues subàrees segons les seves necessitats:

- **Visualització de l'Arbre de Huffman:** Un espai gràfic dedicat a dibuixar l'arbre binari resultant. Això permet observar l'estratègia àvida en acció, mostrant com els símbols més freqüents queden a prop de l'arrel i els menys freqüents s'enfonsen en les branques inferiors. Moltes vegades aquests arbres es poden fer molt grans, i s'han implementat dues barres per poder moure's per el panell que visualitza dit arbre.
- **Diccionari i Entropia:** A la banda dreta, es presenta una taula interactiva amb tres columnes: el símbol identificat, la seva freqüència absoluta dins l'arxiu i el codi binari que se li ha assignat. Just a sobre, un panell d'estadístiques actualitza mètriques crítiques com la *Taxa de Compressió (%)*, el *Temps d'Execució (ms)* i la *Longitud Mitjana* (bits per símbol).

3.3 Panell Inferior (Estat i Seguretat d'Execució)

La part inferior de la interfície compleix una doble funció: donar retroalimentació a l'usuari i protegir l'estat del programa de condicions de carrera (*race conditions*)[2].

- **Feedback visual:** S'hi troba una etiqueta de text que indica l'estat actual al costat d'una barra de progrés i una estimació del temps restant. Aquest element és crucial per a fitxers pesats, evitant que l'usuari percebi que l'aplicació s'ha bloquejat.
- **Bloqueig d'Interfície:** Per garantir la integritat de les dades, s'ha dissenyat un mecanisme que,

en iniciar una operació, bloqueja automàticament tots els botons. D'aquesta manera s'evita que l'usuari pugui carregar un arxiu nou o intentar descomprimir simultàniament mentre el fil d'execució principal està operant.

4 Implementació Tècnica del Model

El cor de la pràctica resideix en el paquet `modelo`, el qual s'ha dissenyat sota el principi de responsabilitat única i eficiència extrema. L'objectiu no ha estat només implementar **Huffman**, sinó crear un sistema capaç de gestionar fitxers binaris i de text de gran envergadura sense comprometre els recursos de la màquina.

4.1 Processament i Gestió de Memòria

Un dels reptes més crítics en el disseny del programari ha estat la gestió de la memòria RAM. Atès que l'aplicació ha de ser capaç de comprimir fitxers que superen la capacitat de la memòria disponible (com els bancs de proves de 10 MB o fitxers d'imatges .bmp), s'ha implementat una arquitectura de fluxos.

- **Lectura mitjançant Fluxos (Streams):** S'utilitza la classe `BufferedInputStream` per llegir el fitxer original en blocs de 8 KB. Aquesta estratègia manté la complexitat espacial constant $O(1)$ respecte a la mida del fitxer N , evitant l'error `OutOfMemoryError` que es produiria si es carregués el fitxer sencer a la memòria.
- **Tractament de Bytes sense Signe:** Java gestiona els bytes amb signe (valors entre -128 i 127). Per indexar correctament l'array de freqüències (0-255), el model aplica una màscara de bits (`0xFF`) a cada lectura. Aquesta operació de baix nivell és essencial per garantir que caràcters amb codis ASCII superiors a 127 es comptabilitzin correctament i no provoquin desbordaments d'índex.
- **Escriptura amb Buffer:** De la mateixa manera, l'escriptura es realitza a través de `BufferedOutputStream`, agrupant les dades abans d'enviar-les al disc per minimitzar el cost temporal de les operacions d'entrada/sortida.

4.2 L'Algorisme Àvid i la Cua de Prioritats

La construcció de l'arbre de Huffman és un exemple clàssic d'algorisme àvid (*greedy*), on es prenen decisions localment òptimes per minimitzar l'entropia global del missatge. En aquest context, l'estratègia consisteix a

combinar iterativament els dos símbols (o subarbres) amb menor freqüència d'aparició.

S'ha definit una interfície genèrica de cua de prioritats (*CuaPrioritat*) per desacoblar l'algorisme de la seva estructura de dades subjacent. Això ens permet intercanviar la implementació de forma dinàmica i comparar-ne el rendiment sense modificar el nucli del compressor de dades.

A continuació tenim les dues estructures de dades que s'han implementat:

- **Binary Heap:** Basat en un munticle binari, ofereix una complexitat de $O(\log n)$ per a les operacions d'inserció i extracció. És l'estructura més equilibrada per al nostre alfabet de $n = 256$ símbols, oferint un rendiment estable i predictable.
- **Fibonacci Heap:** Implementació avançada que permet realitzar insercions en temps constant $O(1)$, posposant el treball de reordenació (consolidació) fins al moment de l'extracció del mínim. Cada node en aquesta estructura manté referències al seu pare, fills i germans en una llista circular doblement enllaçada. En extreure el mínim, es realitza un procés de consolidació que uneix arbres del mateix grau. Tot i la seva superioritat teòrica asimptòtica, per a alfabetes reduïts (256 símbols en el nostre cas), el cost constant de gestionar tants punters fa que no s'observin millores significatives respecte al Binary Heap.[1]

El procediment següent és el següent:

- 1) Inicialitzar la cua de prioritats amb un node fulla per a cada símbol present al fitxer, on la clau de prioritat és la seva freqüència.
- 2) Iterativament, extreure els dos nodes amb menor freqüència (decisió localment òptima).
- 3) Fusionar-los en un nou node pare amb freqüència igual a la suma de les seves freqüències i inserir aquest node de nou a la cua.

Cal destacar que l'elecció de l'estructura de dades per a la cua de prioritats no altera el resultat final de l'arbre (donades unes freqüències fixes), sinó únicament el rendiment del procés de construcció.

5 Gestió de Baix Nivell: Escriptura i Lectura de Bits

La codificació de Huffman genera codis de longitud variable (per exemple, el caràcter 'A' pot ser 0 i el símbol 'P' 101) que no s'alineen amb l'estructura estàndard de 8 bits de la memòria. Per resoldre aquest desajust tècnic, s'ha implementat una capa de gestió de bits manual mitjançant les classes *EscriptorBits* i *LectorBits*.

5.1 Escriptor de Bits

La classe *EscriptorBits* actua com un buffer intermediari entre el model algorísmic i el flux de sortida cap al disc. Atès que el sistema operatiu no pot escriure unitats inferiors a un byte, aquesta classe realitza un empaquetament dinàmic:

- **Buffer d'acumulació:** Es manté un enter intern (el buffer) i un comptador que registra quants bits s'han acumulat fins al moment.
- **Operacions de desplaçament:** Quan el model envia una cadena de bits (com "101"), primer l'escriptor processa cada caràcter de la cadena i, seguidament, desplaça el buffer una posició cap a l'esquerra ($<< 1$) per fer espai al nou bit. Si el bit és un 1, utilitza l'operador lògic `OR(1)` per activar l'últim bit del buffer.
- **Escriptura física:** Només quan el comptador arriba a 8 bits, el buffer es buida (s'aboca) al *BufferedOutputStream* com un byte real i es reinicia el comptador.
- **Gestió del Padding (Finalització):** Al final de la compressió, és probable que l'últim byte no estigui ple (per exemple, només 3 bits ocupats). El mètode `flush()` o `buidar()` desplaça aquests bits cap a l'esquerra per completar un byte de 8 bits (farciment amb zeros), garantint que cap dada es quedi a la memòria RAM.

5.2 Lector de Bits

El procés de descompressió requereix el mecanisme invers, implementat a la classe *LectorBits*. Aquesta classe ha de transformar el flux continu de bytes del fitxer `.huff` en una seqüència de bits individuals que permetin recórrer l'arbre de Huffman.

- **Extracció de bits:** El lector obté un byte sencer del fitxer i utilitza màscares de bits per extreure'n la informació d'un en un. Per exemple, per saber si el bit en la posició i és un 1, s'aplica una operació `AND` lògica.
- **Navegació per l'Arbre:** Cada bit extret actua com una instrucció de moviment per al descompressor: un 0 indica moure's cap al fill esquerre del node actual, i un 1 cap al fill dret.
- **Identificació de Símbols:** El lector continua demanant bits fins que la navegació arriba a un **node fulla**. En aquest moment, s'ha identificat el caràcter original i el procés es reinicia des de l'arrel de l'arbre per al següent caràcter.

5.3 Raonament de disseny

Aquesta implementació bit a bit és el que realment permet la **compressió física** de l'arxiu. Sense aquesta gestió de baix nivell, encara que tinguéssim un arbre de Huffman òptim, ens veuríem obligats a escriure cada codi en un byte sencer, cosa que resultaria en un fitxer de la mateixa mida o fins i tot superior a l'original. L'eficiència s'aconsegueix precisament en "ajuntar" aquests codis de longitud variable dins de l'estructura rígida del byte.

6 Persistència i Descompressió de Dades

6.1 Format del Fitxer Comprimit

El fitxer comprimit no és una simple seqüència de codis, sinó una estructura binària organitzada que garanteix la descompressió sense necessitat d'informació externa.

Aquest format es divideix en dues parts principals:

- **Capçalera:** Ocupa els primers 2.048 bytes del fitxer (256 caràcters $\times$ 8 bytes per cada long). Emmagatzemar les freqüències en lloc de l'arbre sencer redueix la complexitat del format del fitxer. Atès que l'algorisme de construcció de l'arbre és determinista, si el descompressor disposa de les mateixes freqüències, generarà exactament el mateix arbre que el compressor. Immediatament després de la capçalera, s'emmagatzema la mida en bytes del fitxer original. Amb això, aquesta dada és el mecanisme de control que permet al descompressor ignorar els bits de *padding* que s'afegeixen necessàriament a l'últim byte per completar l'estructura de 8 bits del sistema de fitxers.[3]
- **Cos de dades:** El flux de bits empaquetat bit a bit mitjançant la classe `EscriptorBits`.

6.2 Descompressió del Fitxer

La descompressió és un procés de navegació per l'arbre guiat pel flux de bits entrant. La implementació a la classe `LectorBits` i el mètode `descomprimir` de la classe `GestorIO` segueix aquest raonament:

- **Reconstrucció del Model:** Es llegeix la capçalera i s'executa l'instància d'`AlgorismeHuffman` per regenerar l'arbre binari original.
- **Recorregut de l'Arbre:** Per a cada bit llegit del fitxer (mitjançant operacions de màscara AND i desplaçament), el descompressor baixa un nivell en l'arbre: fill esquerre si el bit és 0, fill dret si és 1.

- **Extracció de Símbols:** Quan s'arriba a un node fulla, el símbol recuperat s'escriu al fitxer de sortida i el punter torna immediatament a l'arrel de l'arbre per processar el següent bit.
- **Garantia de Correctesa:** Gràcies a la propietat prefix-free explicada posteriorment, el sistema sap de forma inequívoca quan ha acabat un caràcter i n'ha començat un altre, sense necessitat de caràcters delimitadors que penalitzarien la taxa de compressió.

6.3 Execució Concurrent i Progrés

Per evitar bloquejos de la interfície gràfica, els processos de compressió i descompressió s'executen en fils independents. Això permet mantenir la GUI reactiva i actualitzar en temps real una barra de progrés amb una estimació del temps restant.

A més, s'ha implementat un mecanisme de cancel·lació segura mitjançant una bandera compartida entre fils. Això permet interrompre l'execució de forma controlada sense corrompre els fitxers generats.

La comunicació entre el model i la vista es realitza mitjançant un patró d'escolta (*listener*), garantint un desacoblament adequat.

7 Optimalitat i Correctesa de l'Algorisme

En aquesta secció es justifica per què l'algorisme triat és vàlid per a la compressió i per què garanteix el millor resultat possible.

7.1 Prefix-Free

Perquè la descompressió funcioni sense errors, cap codi pot ser l'inici d'un altre (per exemple, si el caràcter 'A' és 01, cap altre caràcter pot començar per 01...).

Aquesta propietat es garanteix gràcies a l'estructura de l'arbre:

- Tots els caràcters originals es col·loquen exclusivament a les fulles.
- Els nodes interns només serveixen de camí i no contenen informació.

Resultat: Com que cap fulla té descendents, és impossible que un codi sigui prefix d'un altre. Això ens permet llegir bits seguidament sense necessitat d'espais o separadors entre caràcters.

7.2 Optimalitat de l'Algorisme Àvid

L'algorisme de Huffman és "òptim" perquè aconsegueix la longitud mitjana de codi més petita possible per a un fitxer determinat. El seu funcionament es basa en dues lògiques senzilles:

- Nodes menys freqüents al final: Els caràcters que surten poc es col·loquen a la part més profunda de l'arbre (codis llargs), mentre que els més comuns queden a prop de l'arrel (codis curts).
- Decisions òptimes pas a pas: En fusionar sempre els dos nodes amb menor freqüència, l'algorisme assegura que estem minimitzant el cost total de l'arbre de forma progressiva.

7.3 Relació amb l'Entropia

El rendiment del nostre compressor està lligat al concepte d'**Entropia de Shannon**, que marca el límit teòric de quant es pot comprimir un fitxer.

L'algorisme de Huffman s'acosta molt a aquest límit, sovint quedant-se a menys d'un bit de distància del valor ideal.

Exemple pràctic: En les nostres proves amb el text "PATATA", hem comprovat que la longitud mitjana dels codis s'ajusta gairebé perfectament al límit teòric calculat, demostrant que el compressor aprofita al màxim la redundància del text.

8 Anàlisi de Complexitat

Per avaluar l'eficiència del nostre compressor, hem de diferenciar entre el cost del temps (quant tarda) i el cost de l'espai (quanta memòria RAM consumeix).

8.1 Complexitat Temporal

El temps total depèn de la mida del fitxer (N) i del nombre de caràcters diferents (n , que a la pràctica és com a màxim 256).

- **Càlcul de freqüències ($O(N)$):** El programa llegeix el fitxer sencer una vegada. En el nostre fitxer de 10MB, això significa 10 milions d'operacions de lectura.
- **Construcció de l'arbre ($O(n \log n)$):** Com que només tenim un màxim de 256 símbols, aquesta part és instantània (tarda el mateix per a un fitxer d'1KB que per un de 10GB).
- **Compressió/Escriptura ($O(N)$):** Es torna a llegir el fitxer de 10MB per escriure els nous codis.

Conclusió: La complexitat total és $O(N)$. Això vol dir que si el fitxer és el doble de gran, el programa

trigarà el doble de temps. En la nostra pràctica, el factor limitant no és la CPU, sinó la velocitat del disc dur per llegir i escriure aquests 10MB.

8.2 Complexitat Espacial

Aquí és on el disseny de la pràctica destaca per la seva eficiència.

- **Estructures de dades ($O(n)$):** Tant l'arbre com el diccionari i el vector de freqüències ocupen un espai fix (uns pocs KB), ja que el límit són 256 caràcters.
- **Gestió de fluxos ($O(1)$):** Gràcies a l'ús de `BufferedInputStream` i el processament per blocs de 8KB, la memòria RAM no puja mai, independentment de la mida del fitxer. Per exemple: el nostre compressor consumeix la mateixa RAM per comprimir un fitxer de text petit que per a l'arxiu pesat de 10MB o una imatge BMP gran.

9 Cas Pràctic: Funcionament Pas a Pas

Finalment, per il·lustrar empíricament la presa de decisions de l'algorisme àvid i la posterior reducció de l'entropia, presentem un escenari simplificat on l'arxiu original a comprimir conté únicament la paraula "PATATA".

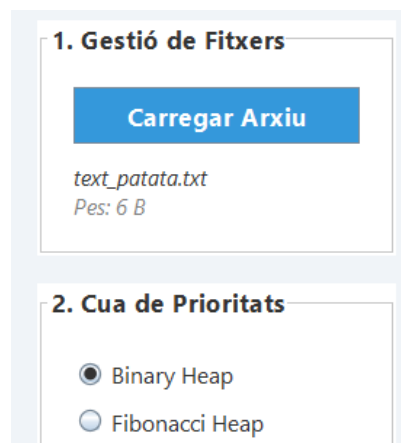


Figura 3. Pes del fitxer 'PATATA'.

9.1 Anàlisi de Freqüències

El primer pas del sistema és llegir els bytes de l'arxiu i calcular-ne la freqüència d'aparició. Per a una cadena de 6 caràcters (ocupant inicialment $6 \times 8 = 48$ bits en codificació ASCII), el resultat és el següent:

Taula 1
Taula de freqüències inicials

Símbol	Aparicions	Probabilitat
A	3	50.0 %
T	2	33.3 %
P	1	16.7 %

9.2 Construcció Àvida de l'Arbre

La nostra cua de prioritats (*Min-Heap*) s'inicialitza amb tres nodes fulla: (P:1), (T:2), (A:3). L'algorisme aplica el criteri de curta mira iterativament:

- 1) **Iteració 1:** S'extreuen els dos nodes amb menor freqüència absoluta: (P:1) i (T:2). Es fusionen creant un node intern pare amb freqüència $1 + 2 = 3$. Aquest nou node (N1:3) s'insereix de nou a la cua.
- 2) **Iteració 2:** La cua conté ara (N1:3) i (A:3). S'extreuen i es fusionen per crear la **Root** (arrel) amb freqüència total $3 + 3 = 6$. Com que només queda un node a la cua, l'arbre s'ha completat.

A continuació es mostra l'arbre resultant:

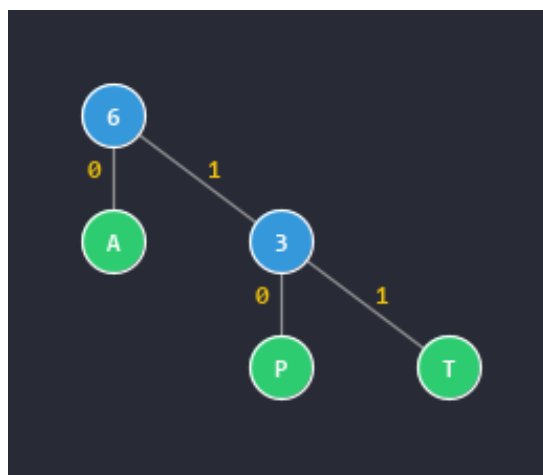


Figura 4. Arbre resultant de la paraula 'PATATA'.

9.3 Generació del Diccionari i Resultats

En recórrer l'arbre des de l'arrel fins a les fulles (assignant '0' a la branca esquerra i '1' a la dreta), obtenim els codis binaris òptims. S'observa com la lletra 'A', sent la més freqüent, rep el codi més curt (1 sol bit), confirmant l'èxit de l'algorisme.

Utilitzant aquest diccionari, la paraula "PATATA" es tradueix al flux de bits: 10 0 11 0 11 0. L'impacte sobre el pes de l'arxiu és el següent:

- **Pes Original:** 6 caràcters $\times$ 8 bits = 48 bits.

Taula 2
Diccionari de Codis de Huffman resultant

Símbol	Codi Binari	Longitud del codi
A	0	1 bit
P	10	2 bits
T	11	2 bits

- **Pes Comprimit:** $(3 \times 1) + (2 \times 2) + (1 \times 2) = 9$ bits.

Això suposa una **longitud mitjana** d'1,5 bits per símbol (molt inferior als 8 bits originals) i una **taxa de compressió matemàtica** del 81,25%, calculada mitjançant la fórmula:

$$Taxa(\%) = \left(1 - \frac{9}{48}\right) \times 100$$

Aquest exemple a petita escala demostra que les decisions locals preses en cada pas per l'algorisme (ajuntar els menors) deriven directament en un resultat global òptim on l'espai d'emmagatzematge s'aprofita al màxim en funció de la naturalesa de les dades.

Estadístiques

Taxa de compressió: 81,25 %

Temps d'execució: 4 ms

Longitud mitjana: 1,50 bits/símbol

Diccionari / Entropia

Símbol	Freqüència	Codi Huffman
'P'	1	10
'A'	3	0
'T'	2	11

Figura 5. Estadístiques de la paraula 'PATATA'.

S'ha observat que en fitxers extremadament petits (com el cas de la paraula 'PATATA'), la taxa de compressió és negativa si afegim la capçalera. Això es deu al fet que la capçalera de 2 KB necessària per fer el fitxer (guardar les freqüències i la mida original) és molt superior a la mida del missatge original. Per tant per solucionar-ho hem calculat tan sols el cost pur de la codificació del fitxer sense capçalera.

9.4 Aplicació de la Fibonacci Heap

El Fibonacci Heap és una estructura avançada que permet inserir elements en temps constant $O(1)$, posposant la reordenació fins al moment d'extreure el mínim. Tot i la seva superioritat teòrica, el seu ús en aquesta pràctica ens porta a conclusions interessants.

L'exemple de "PATATA":

Si analitzem el cas de la paraula "PATATA", observem per què el Fibonacci Heap no marca la diferència: Només tenim 3 caràcters diferents ('P', 'A', 'T'), l'algorisme només ha de fer un parell de fusions per crear l'arbre i el Fibonacci Heap necessita gestionar llistes circulars i molts punters per a cada node. En un cas tan petit, triga més temps a "preparar" l'estructura que el que estalvia en l'execució.

Estadístiques

Taxa de compressió: 81,25 %

Temps d'execució: 5 ms

Longitud mitjana: 1,50 bits/símbol

Diccionari / Entropia

Símbol	Freqüència	Codi Huffman
'P'	1	10
'A'	3	0
'T'	2	11

Figura 6. Estadístiques de la paraula 'PATATA' amb Fibonacci Heap.

Conclusions de la comparativa: El Binary Heap és més senzill i ràpid per a alfabet petits (com els 256 caràcters de l'ASCII). El Fibonacci Heap té un "cost constant" molt alt: moure tants punters de memòria penalitza el rendiment en execucions curtes. A més, el Fibonacci Heap només seria clarament superior si treballéssim amb milions de símbols diferents, on les insercions $O(1)$ compensarien el cost de la consolidació posterior.

Resultat en la pràctica: Per al nostre compressor, el Binary Heap és més eficient. El temps d'execució depèn del nombre de símbols (n) i no de la mida del fitxer (N); per tant, processar un fitxer de 10MB amb la paraula 'PATATA' tarda pràcticament el mateix amb una estructura o l'altra, ja que l'arbre es construeix només una vegada al principi.

9.5 Altres exemples

A continuació mostrem un altre exemple per un fitxer en format d'imatge comprimida en format .png, en la qual podem observar una taxa de compressió mínima, d'un 0.8%, i una longitud mitjana de 7.9 bits/símbol, això es així perquè aquest format ja està prèviament comprimit.



Figura 7. GUI per execució d'una imatge .png.

En canvi, per una imatge sense comprimir en format .bmp, la taxa de compressió és molt més elevada, estem parlant d'un 50% enlloc d'un 0.8% en funció del format; a més d'una longitud mitjana de 3.8 bits/símbol.



Figura 8. GUI per execució d'una imatge .bmp.

Per últim, cal recalcar que tots els arbres es guarden automàticament a una carpeta del directori anomenada imatgesArbres per una millor visualització.

10 Conclusió

La realització d'aquesta pràctica ens ha servit per entendre que, en el món real, un bon algorisme no és res sense una implementació eficient. A diferència d'altres projectes, el compressor de Huffman ens ha obligat a anar més enllà de la teoria i a gestionar recursos crítics del sistema, com el flux de dades a baix nivell i l'ús de la memòria RAM.

Durant el desenvolupament, hem pogut constatar la potència de l'estratègia àvida. És sorprenent com una lògica tan senzilla com la de fusionar els nodes amb menor freqüència pot arribar a generar una solució òptima per a la transmissió d'informació. Això sí, l'exemple de la paraula "PATATA" ens ha tocat de peus a terra: ens ha ensenyat que la compressió té un peatge inevitable, l'overhead de la capçalera, i que en fitxers molt petits aquest cost pot superar el benefici de l'algorisme.

Un dels punts que més ens ha fet reflexionar ha estat la comparativa entre el Binary Heap i el Fibonacci Heap. Ens ha servit de lliçó d'enginyeria: la superioritat teòrica de la complexitat asimptòtica no sempre guanya a la pràctica. Hem après que, per a alfabets de 256 caràcters, una estructura més simple i amb menys "soroll" de punters com el Binary Heap és molt més eficient que una de més complexa com la de Fibonacci.

En definitiva, aquest projecte ens ha donat una visió "extrem a extrem" del que és el desenvolupament de programari: des de barallar-nos amb màscares de bits fins a dissenyar una interfície gràfica que no es bloquegi gràcies a la concurrència. Estem molt satisfets amb el resultat final, ja que hem aconseguit un sistema robust i capaç de gestionar fitxers pesats amb un consum de memòria totalment estable, complint amb els estàndards de qualitat que demana l'assignatura d'Algorismes Avançats.

Referències

- [1] Assignatura d'Algorismes Avançats, Universitat de les Illes Balears, 2026.
- [2] *Gemini*, model 3.5 Pro, Google
- [3] Codificació Huffman, Viquipèdia
- [4] Oracle Corporation, "Concurrency in Swing," *The Java Tutorials*. [En línia]. Disponible a: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/index.html>.